LANGCHAIN COMPLETE GUIDE FOR ABSOLUTE BEGINNERS

Understanding EVERY concept from scratch with practical examples
Works with Gemini, OpenAI, or any other AI model

CHAPTER 1: WHAT IS LANGCHAIN?

🤔 What is LangChain?

Imagine you want to build an AI application. You need to:

- Connect to AI models (like ChatGPT, Gemini)
- Send organized questions/prompts
- Get structured responses
- Chain multiple AI operations together
- Remember conversation history

LangChain is a **FRAMEWORK** that makes all this **EASY**!

Real-World Analogy

- **Without LangChain:** Building a house with bare hands (very hard!)
- **With LangChain:** Building a house with tools and blueprints (easy!)

Think of LangChain as a **TOOLKIT** for AI applications.

CHAPTER 2: INSTALLATION - Setting up your environment

📦 What We're Installing

- **langchain:** Main framework (the toolkit)
- **langchain-google-genai:** Connection to Google's Gemini AI
- **langchain-openai:** (Optional) Connection to OpenAI's ChatGPT
- **google-generativeai:** Google's AI library

Why So Many Packages?

- **langchain** = Core tools
- **langchain-google-genai** = Specific connector for Gemini
- Different AI models need different connectors

CODE: Installation and Setup

```
# Step 1: Install packages
!pip install -q langchain langchain-google-genai google-generativeai langchain-
```

```
community langchain-openai

# Step 2: Import what we need
import os
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import PromptTemplate, ChatPromptTemplate
from langchain.schema import HumanMessage, SystemMessage, AIMessage
from langchain.output_parsers import StructuredOutputParser, ResponseSchema
from pydantic import BaseModel, Field
from typing import List

# Step 3: Set up API key (Get from: https://makersuite.google.com/app/apikey)
os.environ["GOOGLE_API_KEY"] = "YOUR_API_KEY_HERE"  # ⚠️ REPLACE THIS!
print("✅ Installation Complete!\n")
```

CHAPTER 3: UNDERSTANDING LLMs (Large Language Models)

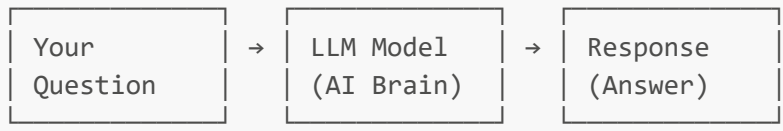
🧠 What is an LLM?

LLM = **Large Language Model** (AI that understands and generates text)

Think of it like this:

- **You:** "What is 2+2?"
- **LLM Brain:** *processes question*
- **LLM:** "2+2 equals 4"

How It Works (Visual Flow)



Parameters You Need to Know

- **model:** Which AI brain to use ("gemini-1.5-flash", "gpt-4", etc.)
- **temperature:** How creative the AI should be
  - **0.0** = Very focused, predictable (for facts)
  - **1.0** = Very creative, random (for stories)

EXAMPLE 1: Creating and Using an LLM

```
# Step 1: Initialize the AI model
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",  # Which AI model to use
    temperature=0.7,  # Creativity level (0-1)
```

```
    convert_system_message_to_human=True  # Technical setting for Gemini
)

# Step 2: Ask a simple question
response = llm.invoke("Explain what Python is in one sentence")

# Step 3: Print the response
print("AI Response:", response.content)
```

CHAPTER 4: UNDERSTANDING TEMPERATURE (Creativity Control)

🎚️ What is Temperature?

Temperature controls how **CREATIVE** or **FOCUSED** the AI is.

Visual Representation

```
Temperature 0.0 ──────────────────── Temperature 1.0
  (Focused)                              (Creative)
     ↓                                      ↓
"Python is a                  "Python is a magical
programming language"         wizard's spellbook for
                              commanding computers!"
```

When to Use What

- **Temperature 0.0-0.3:** Facts, data analysis, math
- **Temperature 0.4-0.7:** Balanced responses, general use
- **Temperature 0.8-1.0:** Creative writing, brainstorming

EXAMPLE 2: Temperature Comparison

```
# Low temperature (focused)
llm_focused = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0.0  # Very focused
)

# High temperature (creative)
llm_creative = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=1.0  # Very creative
)

question = "Give me a name for a coffee shop"
print("FOCUSED (temp=0.0):", llm_focused.invoke(question).content)
print("CREATIVE (temp=1.0):", llm_creative.invoke(question).content)
```

CHAPTER 5: MESSAGES - Different Types of Conversations

💬 Understanding Message Types

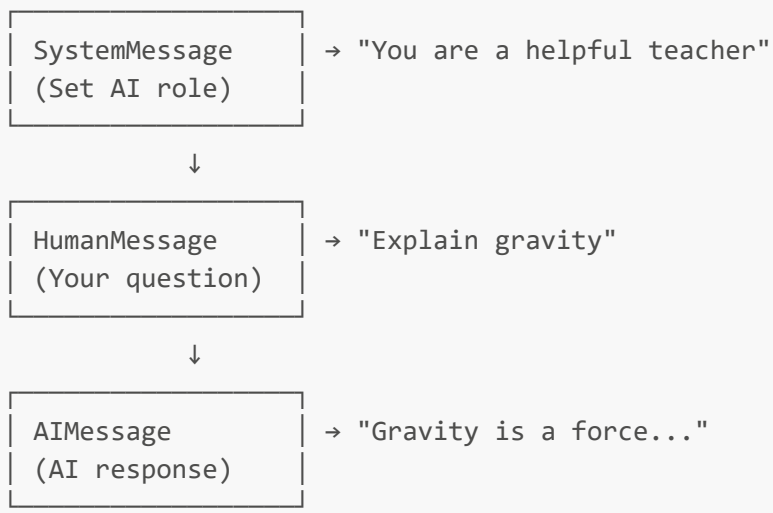
When talking to AI, we use different message types:

1. **SystemMessage:** Instructions/Rules for AI (AI's behavior/role)
2. **HumanMessage:** Your questions/input
3. **AIMessage:** AI's previous responses (used for memory/context)

Real-World Analogy

- **SystemMessage:** "You are a professional chef assistant"
- **HumanMessage:** "How do I make pasta?"
- **AIMessage:** "First, boil water..." (assistant's previous response)

Visual Flow



EXAMPLE 3: Using Different Message Types

```
# Create a conversation with specific role
messages = [
    SystemMessage(content="You are a friendly math tutor for kids. Use simple
words."),
    HumanMessage(content="What is multiplication?")
]

response = llm.invoke(messages)
print("AI Response:", response.content)
```

CHAPTER 6: PROMPT TEMPLATES - Reusable Question Formats

What are Prompt Templates?

Problem: Writing the same prompt structure again and again is boring!

A **Prompt Template** is a defined, reusable format with placeholders (variables) for dynamic input.

Analogy

Like a form with blanks to fill: **"Dear {name}, you have won {prize}!"**
Just change 'name' and 'prize' each time!

Structure

```
Template → Fill Variables → Complete Prompt → Send to AI
```

EXAMPLE 4: Creating and Using Prompt Templates

```
# Step 1: Create a template with variables in {}
template = """
You are a {profession} expert.
Explain {topic} to a {audience} in simple terms.
Keep it under 3 sentences.
"""

# Step 2: Create PromptTemplate object
prompt = PromptTemplate(
    input_variables=["profession", "topic", "audience"],  # Variables to fill
    template=template  # The template text
)

# Step 3: Fill the template with actual values
filled_prompt = prompt.format(
    profession="biology",
    topic="photosynthesis",
    audience="10-year-old child"
)

print("Filled Prompt:", filled_prompt)
print("\nAI Response:", llm.invoke(filled_prompt).content)

# Step 4: Use same template with different values
filled_prompt2 = prompt.format(
    profession="computer science",
    topic="algorithms",
    audience="beginner programmer"
)
```

```
print("Different Values:")
print("AI Response:", llm.invoke(filled_prompt2).content)
```

CHAPTER 7: CHAINS - Connecting Prompt Template + LLM

What are Chains?

A **Chain** is the mechanism for connecting multiple steps or components together. The most basic chain connects a **Prompt Template** to an **LLM**.

Simple Chain Flow

```
Step 1: Prompt Template → Step 2: LLM → Step 3: Output
```

Why Use Chains?

Chains simplify the workflow. Instead of three manual steps:

1. Create template
2. Fill template
3. Send to LLM

You create the connection once: `chain = template | llm`, and then just run: `chain.invoke(data)`.

EXAMPLE 5: Creating Your First Chain

```
# Step 1: Create prompt template
prompt_template = PromptTemplate(
    input_variables=["animal"],
    template="Tell me 3 interesting facts about {animal}. Be brief."
)

# Step 2: Create chain (connect template to LLM)
chain = prompt_template | llm  # The | symbol connects them!

# Step 3: Use the chain (so simple!)
response = chain.invoke({"animal": "dolphins"})
print("Response:", response.content)
```

CHAPTER 8: SEQUENTIAL CHAINS - Multi-Step Processing

What are Sequential Chains?

A **Sequential Chain** runs multiple chains or steps in sequence, where the **output of one step becomes the input for the next step**.

Flow Diagram

```
Input → Chain 1 → Output 1 → Chain 2 → Output 2 → Chain 3 → Final Output
```

Real-World Example: Writing a Story

1. **Chain 1:** Generate story idea (Theme → Idea)
2. **Chain 2:** Expand into paragraph (Idea → Full Story)

EXAMPLE 6: Sequential Chain - Story Writer

```
# Chain 1: Generate topic
prompt1 = PromptTemplate(
    input_variables=["theme"],
    template="Generate a creative one-sentence story idea about {theme}."
)
chain1 = prompt1 | llm

# Chain 2: Expand into paragraph
prompt2 = PromptTemplate(
    input_variables=["idea"],
    template="Expand this story idea into a detailed paragraph:\n{idea}"
)
chain2 = prompt2 | llm

# Execute sequential chains manually
print("Step 1: Generate Idea")
idea_response = chain1.invoke({"theme": "robots and friendship"})
idea = idea_response.content
print("Idea:", idea)

print("\nStep 2: Expand Idea")
story_response = chain2.invoke({"idea": idea})  # Output of Chain 1 is input for
Chain 2
print("Full Story:", story_response.content)
```

CHAPTER 9: OUTPUT PARSERS - Getting Structured Data

🎯 What are Output Parsers?

Problem: AI returns **TEXT**, but our code often needs **STRUCTURED DATA** (JSON, lists, objects).

An **Output Parser** is a tool that converts the AI's raw text response into a reliably structured format.

Flow

```
AI Response (text) → Parser → Structured Data (JSON/Object)
```

Why Important?

- Easy to use in code (e.g., accessing `parsed_data['name']`)
- Can save directly to a database
- Ensures consistency for calculations

EXAMPLE 7: Using Structured Output Parser

```
# Step 1: Define what structure we want
response_schemas = [
    ResponseSchema(name="name", description="Name of the country"),
    ResponseSchema(name="capital", description="Capital city"),
    ResponseSchema(name="population", description="Population in millions"),
    ResponseSchema(name="language", description="Official language")
]

# Step 2: Create parser
parser = StructuredOutputParser.from_response_schemas(response_schemas)

# Step 3: Create prompt with parser instructions
prompt = PromptTemplate(
    template="Provide information about {country}.\n{format_instructions}",
    input_variables=["country"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

# Step 4: Create chain
chain = prompt | llm

# Step 5: Get structured output
response = chain.invoke({"country": "Japan"})
parsed_data = parser.parse(response.content)

print("Structured Data:", parsed_data)
print(f"Capital: {parsed_data['capital']}")  # Easy access to structured fields
```

CHAPTER 10: PYDANTIC MODELS - Advanced Data Structures

🏗️ What is Pydantic?

Pydantic is a Python library for **data validation** and structured settings management, often used for its **TYPE CHECKING** capabilities.

Comparison

- **Regular Dict:** `{name: "John", age: "30"}` ← Age is a string (potential mistake!)
- **Pydantic:** `{name: "John", age: 30}` ← Age is enforced as an integer (correct!)

Benefits

1. **Type Safety:** Ensures data types are correct (e.g., `int`, `float`, `str`)
2. **Automatic Validation:** Pydantic handles checking the data
3. **Clear Structure:** Uses Python classes (`BaseModel`) for a clean definition

EXAMPLE 8: Pydantic Output Parser (Movie Details)

```
# Step 1: Define data structure with types
class Movie(BaseModel):
    title: str = Field(description="Movie title")
    director: str = Field(description="Director name")
    year: int = Field(description="Release year")
    genre: str = Field(description="Movie genre")
    rating: float = Field(description="Rating out of 10")

# Step 2: Create Pydantic parser
from langchain.output_parsers import PydanticOutputParser
pydantic_parser = PydanticOutputParser(pydantic_object=Movie)

# Step 3: Create prompt
prompt = PromptTemplate(
    template="Generate details for a {genre} movie.\n{format_instructions}",
    input_variables=["genre"],
    partial_variables={"format_instructions":
pydantic_parser.get_format_instructions()}
)

# Step 4: Execute
chain = prompt | llm
response = chain.invoke({"genre": "sci-fi"})
movie = pydantic_parser.parse(response.content)

print(f"Title: {movie.title}")
print(f"Rating: {movie.rating}/10")
```

CHAPTER 11: COMPLETE PROJECT - Restaurant Menu Generator

🍽 Putting It All Together

This project combines all core LangChain components:

- ✓ **LLM** (AI model)
- ✓ **Prompt Templates** (reusable prompts)
- ✓ **Pydantic** (structured output)
- ✓ **Chains** (connect everything)

EXAMPLE 9: Complete Restaurant Menu Generator

```
# Define menu structure using Pydantic
class MenuItem(BaseModel):
    name: str = Field(description="Dish name")
    description: str = Field(description="Brief description")
    price: float = Field(description="Price in USD")

class RestaurantMenu(BaseModel):
    restaurant_name: str = Field(description="Restaurant name")
    cuisine: str = Field(description="Type of cuisine")
    appetizers: List[MenuItem] = Field(description="List of 2 appetizers")
    main_courses: List[MenuItem] = Field(description="List of 2 main courses")
    desserts: List[MenuItem] = Field(description="List of 2 desserts")

# Create parser
menu_parser = PydanticOutputParser(pydantic_object=RestaurantMenu)

# Create prompt
menu_prompt = PromptTemplate(
    template="""Create a restaurant menu for a {cuisine} restaurant.
    Dietary preference: {dietary_preference}
    Price range: {price_range}

    {format_instructions}
    """,
    input_variables=["cuisine", "dietary_preference", "price_range"],
    partial_variables={"format_instructions":
menu_parser.get_format_instructions()}
)

# Create chain
menu_chain = menu_prompt | llm

# Generate menu
response = menu_chain.invoke({
    "cuisine": "Italian",
    "dietary_preference": "vegetarian",
    "price_range": "moderate ($10-$25)"
})
menu = menu_parser.parse(response.content)

# Display menu
print(f"\n{'='*70}")
print(f"🍽️ {menu.restaurant_name}")
print(f"{'='*70}")
print(f"Cuisine: {menu.cuisine}\n")

print("🥗 APPETIZERS:")
for item in menu.appetizers:
    print(f" • {item.name} - ${item.price}")
    print(f"   {item.description}\n")
```

```
print("🍝 MAIN COURSES:")
for item in menu.main_courses:
    print(f" • {item.name} - ${item.price}")
    print(f"   {item.description}\n")

print("🍰 DESSERTS:")
for item in menu.desserts:
    print(f" • {item.name} - ${item.price}")
    print(f"   {item.description}\n")
```

CHAPTER 12: WORKING WITH OTHER AI MODELS

🔄 How to Use with Other AI Models

The **SAME** concepts (**Templates**, **Chains**, **Parsers**) work with **ANY** AI model!

This is LangChain's primary strength: **Model Agnosticism**. It acts as a standardized API layer between your code and the AI provider.

Diagram: LangChain's Universal Interface



Examples: Changing the LLM

GEMINI (Google):

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash")
```

OPENAI (ChatGPT):

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4")
```

ANTHROPIC (Claude):

```
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-3-sonnet")
```

Universal Template - Works with ANY AI Model

```
# This code structure works with Gemini, OpenAI, Claude, etc.
universal_template = PromptTemplate(
    input_variables=["task"],
    template="Help me with: {task}"
)

universal_chain = universal_template | llm  # llm can be ANY model!
response = universal_chain.invoke({"task": "explain quantum computing simply"})
print(response.content)
```

CHAPTER 13: KEY CONCEPTS SUMMARY

Complete Summary of All Concepts

LLM (Large Language Model)

- **What:** AI brain that processes text
- **Why:** To get AI-powered responses

Temperature

- **What:** Controls creativity (0=focused, 1=creative)
- **Why:** Different tasks need different creativity levels

Messages (System, Human, AI)

- **What:** Different types of conversation messages
- **Why:** To structure conversations and set AI behavior

Prompt Templates

- **What:** Reusable prompt structures with variables
- **Why:** Avoid writing same prompts repeatedly

Chains

- **What:** Connect template + LLM together (using the | operator)
- **Why:** Simplify workflow (one line instead of many)

Sequential Chains

- **What:** Multiple chains in sequence
- **Why:** For multi-step tasks (output of one step feeds into the next)

Output Parsers

- **What:** Convert AI text to structured data (JSON/objects)
- **Why:** Easy to use in code/database/UI

Pydantic

- **What:** Type-safe data structures for parsers
- **Why:** Validation and complex data handling

Universal Approach (Model Agnosticism)

- **What:** Same chain/template code works with any AI model
- **Why:** Easy to switch between models for cost or performance reasons

Learning Path

- **Day 1:** LLM, Temperature, Messages
- **Day 2:** Prompt Templates, Chains
- **Day 3:** Output Parsers, Pydantic
- **Day 4:** Build complete projects

☑ COMPLETE TUTORIAL FINISHED!

Next Steps

1. Replace **'YOUR_API_KEY_HERE'** with your Gemini API key
2. Run each example one by one
3. Modify examples with your own ideas
4. Try combining concepts in new ways

Practice Projects

- Email Writer (with tone options)
- Study Guide Generator
- Recipe Recommender
- Travel Planner
- Code Explainer

Remember: Practice makes perfect! Start simple, then build complexity.